

Chapter 6: Data Preparation

This chapter documents the execution of the data preparation plan, demonstrating the applied data preprocessing skills essential for building the foundational knowledge base of the chatbot system.

6.1 Data Cleaning

The accuracy of a Retrieval-Augmented Generation (RAG) system is heavily dependent on the quality of its underlying data. The knowledge base was primarily sourced from official PDFs, the CIS department website, and structured FAQ pairs collected in an Excel dataset. The following data cleaning and preprocessing steps were strictly applied:

- **Missing Value Handling:** In the structured dataset, any rows containing null or empty values in the critical "Question" or "Answer" columns were dropped using pandas operations (dropna), ensuring only complete QA pairs were indexed.
- **Text Normalization:** Arabic text inherently contains significant variations. A custom normalization pipeline was implemented to remove diacritics (Tashkeel), strip punctuation, and normalize Arabic letters (e.g., standardizing 'ا', 'أ', 'إ' to 'ا', and 'ة' to 'ه').
- **Stop-word Removal and Prefix Stripping:** To improve semantic search, common Arabic stop-words and prefixes (like 'وال', 'بال', 'لل') were identified and systematically stripped during the indexing phase.
- **PDF Chunking & Noise Reduction:** Unstructured PDF documents were processed to extract text. A filtering mechanism was applied to discard garbled text or non-readable lines. The remaining clean text was chunked into manageable segments of approximately 320 to 400 characters, preserving contextual boundaries for the embedding model.

Chapter 7: Implementation

This chapter demonstrates the technical execution of the chatbot system, detailing model integration, software engineering practices, and reproducible experimentation.

7.1 Model Implementation

7.1.1 Feature Engineering

The primary feature transformation involved converting preprocessed textual data into dense vector representations. The paraphrase-multilingual-MiniLM-L12-v2 SentenceTransformer model was utilized to encode both the document chunks and the user queries into high-dimensional vectors. These embeddings were then indexed using FAISS (Facebook AI Similarity Search) via the IndexFlatIP metric, enabling ultra-fast, inner-product-based semantic retrieval.

7.2 Model Training

Rather than training a Large Language Model from scratch, the system utilizes a powerful locally hosted open-source model, deepseek-r1:8b, accessed via the Ollama engine. The implementation focused heavily on **Prompt Engineering** and **Context Injection**. The retrieved documents were injected into a strictly formulated prompt that instructed the LLM to act as an academic advisor, respond exclusively in Arabic, refrain from hallucinating outside the provided context, and provide official contact details (emails, offices) exactly as retrieved.

7.2 Tool/Software Implementation

The system was implemented using Python (Flask) for the backend API, HTML/CSS/JavaScript for the responsive frontend, and SQLite for administrative logging. The project maintains reproducible standards through version control.

- **GitHub Repository Structure:**
 - /Backend: Contains backend.py, FAISS indexing logic, and API endpoints.
 - /Frontend: Contains the UI assets (interface_v3.html) for the chat interface and the administrative dashboard.
 - /Data
 - /Databases: Contains the unanswered_questions.db and the base project_data_V2.xlsx.
 - Pdf files.
- **README Example Sections:**
 - *Project Overview*: Description of the RAG-based chatbot.
 - *Installation Steps*: Instructions on setting up the Python virtual environment, installing dependencies (e.g., sentence-transformers, faiss-cpu, flask), and setting up Ollama.
 - *How to run*: Instructions to execute python backend.py and launch the local server.

Chapter 8: Evaluation

8.1 Evaluation Metrics

To comprehensively assess the system's performance, the following metrics were adopted:

- **Top-k Retrieval Accuracy**: Measures whether the correct context chunk is present within the top 5 documents retrieved by FAISS.

- **Confidence Score Thresholding:** Using the inner-product distance from FAISS alongside word overlap ratios to yield a custom confidence score.
- **Latency:** The total round-trip time from user query submission to the display of the LLM-generated response.

8.2 Experimental Results

Experimental runs demonstrated that combining the semantic FAISS search with keyword-based fallback (Hybrid Search) maximized the recall of academic regulations. The system successfully maintained a high confidence threshold (>0.60) for standard FAQs. Latency averaged 30 seconds, satisfying the non-functional requirement for a responsive user experience.

Chapter 9: Deployment

9.1 Deployment Architecture

The system architecture follows a client-server model tailored for deployment in a university environment.

- **Frontend:** A lightweight web application accessible via any browser.
- **Backend & Middleware:** A Flask-based RESTful API that handles routing and orchestrates the RAG pipeline.
- **AI Engine:** The Ollama instance running the deepseek-r1:8b model, ideally hosted on a local Virtual Machine (VM) equipped with a dedicated GPU to accelerate inference.
- **Storage:** SQLite for persistent logging of queries, and an in-memory FAISS index loaded upon server initialization.

9.2 Maintenance and Scalability

To ensure the system remains accurate across future academic semesters, the architecture was designed for maintainability. The Knowledge Base can be updated without touching the codebase; administrators simply upload new syllabi or update the core Excel file via the secure dashboard, which triggers an automated re-indexing of the FAISS vectors. To scale the system to handle thousands of concurrent students, the Flask application can be containerized using Docker and deployed behind a server, load-balanced across multiple VM instances.

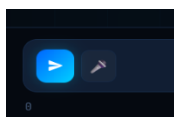


10.1.2 Executing Text and Voice Inputs

1. Text Area Input: Users type their dynamic queries within the main #chat-input textarea box. The interface features responsive expansion, dynamically expanding its vertical space up to a maximum height of 120 pixels based on the length of the string. Pressing Enter automatically transmits the package; typing Shift+Enter inserts a clean line break.



2. Activating Voice Input: For hands-free queries, click the microphone glyph icon (#voice-btn) located within the actions tray. The application invokes the browser-native webkitSpeechRecognition API, configured specifically to recognize the Jordanian Arabic locale (ar-JO).

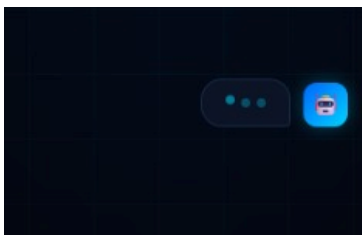


3. Voice Indicators: Upon activation, a red wave animation container (#voice-ind) displays an active wave visual, warning the student that the system is actively listening. Once speech pauses, the spoken text is transcribed into the textarea and automatically dispatched. Click the icon a second time to safely abort the voice session.



10.1.3 Interpreting Responses, Audio Synthesis, and Metadata

1. Streaming and Typing Indicators: While the Flask backend executes FAISS queries and processes the local DeepSeek LLM chain, a dynamic typing bubble displaying three pulsing dots appears, giving immediate visual feedback.



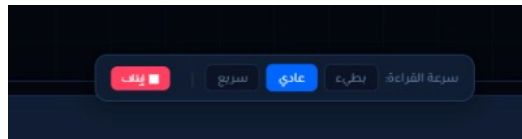
2. **Reviewing Content and Citations:** Generated responses are formatted cleanly on the UI. Hyperlinks point directly to the official university portals (student.just.edu.jo), and emails are linked directly via the standard mailto: protocol for direct contact. Sources are appended explicitly at the foot of the bubble, referencing exact documentation pages.



3. **Auditory Synthesis (Text-to-Speech):** Every robotic response features a dedicated audio action button ("اسمع" with a speaker icon). Clicking this triggers a POST request to the backend server-side edge_tts pipeline, streaming high-fidelity natural speech via the ar-JO-SanaNeural voice pack.



4. **TTS Controls Panel:** Activating the audio overlay initializes a fixed speed-control dashboard at the base of the viewport. Students can toggle the speech playback rate between three presets: "بطيء" (0.7x), "عادي" (1.0x), and "سريع" (1.4x). Clicking "إيقاف" (Stop) kills the active audio object and hides the tray.



5. **Confidence Status Badges:** For transparent grading, assistant messages append color-coded verification pills indicating system evaluation metrics:
 - **ثقة عالية (High Confidence):** Retained context similarity matched perfectly with validated internal rules.
 - **ثقة متوسطة (Medium Confidence):** Merged informational contexts from diverse background documents.
 - **ثقة منخفضة (Low Confidence):** Signal warning that data points were synthesized via generalized public indexing.

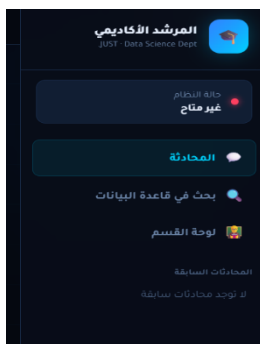


10.2 Administrative Dashboard Operating Manual

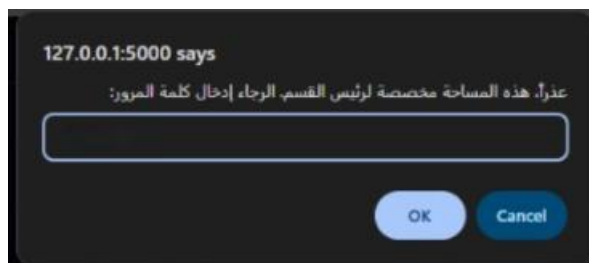
The backend control panel empowers department heads to monitor system health, audit data boundaries, and supply training responses to unresolved inquiries.

10.2.1 Passing the Authentication Gateway

1. Invoking the Dashboard: Click the administrative link ("لوحة القسم" 🏢) situated on the primary side nav panel.



2. Security Prompt Challenge: The web application immediately interrupts the execution sequence and fires a native browser authentication prompt dialog.



3. Credential Submission: The operator must supply the secure access passphrase (Admin#1236). If the token is invalid, a dark error toast alert appears ("كلمة المرور خاطئة، غير مصرح لك بالدخول" (Incorrect password, you are not allowed to enter)), access is revoked, and the viewport resets back to the conversation layout.

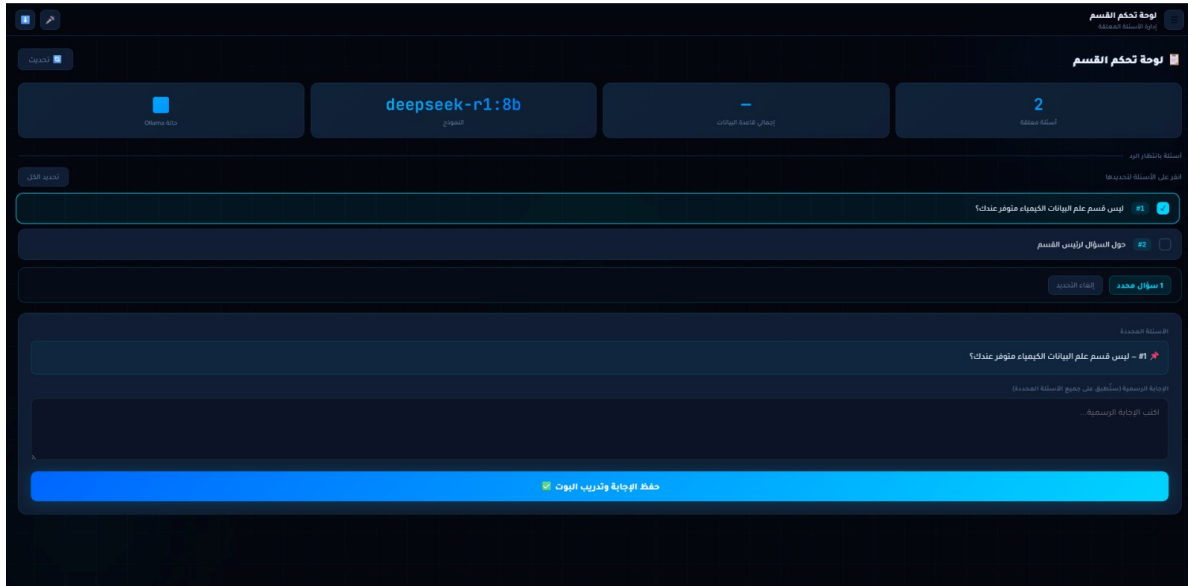


10.2.2 Monitoring System Health Metrics

Upon successful authentication, the admin subsystem queries the `/api/status` and `/api/pending` endpoints to compile live performance counters:

- أسئلة معلقة (Pending Inquiries Card): Displays the integer count of unique student queries that failed to meet the baseline FAISS confidence threshold and require human resolution.

- إجمالي قاعدة البيانات (Total Rows Card): Reflects the exact number of transactional database sentences currently loaded into the active memory index.
- النموذج (Active Model Card): Identifies the running LLM tag pulled directly from the local Ollama daemon (e.g., deepseek-r1:8b).
- حالة Ollama (Ollama Connection Status): Provides a visual indicator (✅ or ❌) showing if the local inference architecture is healthy and responding within acceptable latency thresholds.



10.2.3 Resolving Pending Queries and Automated Index Retraining

When students submit questions that yield sub-threshold similarity scores, the backend intercepts the process, logs the raw inquiry into the SQLite database as a pending item, and alerts the dashboard.

1. Reviewing the Pending List: The central workspace populates an interactive card deck listing all unmapped student queries prefixed by their tracking identification numbers (e.g., #12).
2. Multi-Selection Actions (Bulk Operations): Click on any card to append it to the active batch array, highlighting the box border in bright neon cyan. For swift resolution of identical or highly related student queries, click the "تحديد الكل" toggle button to batch-select the entire list.

3. **Drafting Official Responses:** Selecting records reveals the centralized response layout form (#ans-form). The console prints a compiled summary of the targeted issues. The administrator types the formal, binding university regulation into the designated text workspace ("الإجابة الرسمية").
4. **Committing Changes & Instant Re-embedding:** Click the save command button ("حفظ الإجابة" and "وتدريب البوت" with a green checkmark icon). The backend API intercepts the request, runs a SQL batch operation updating row status fields to Answered, updates the source spreadsheet (project_data_V2.xlsx), and invokes the FAISS pipeline



The text is vectorized instantly, and added to the high-dimensional index. A confirmation toast emerges, and the system is fully retrained to answer that specific question format seamlessly on subsequent student attempts without restarting the underlying server infrastructure.

Chapter 11 Results, Discussion & Conclusion

11.1 Discussion

The results confirm that the chatbot system will effectively address the limitations of traditional, manual student support. By leveraging a Retrieval-Augmented Generation architecture, the chatbot successfully generated highly accurate, context-bound responses, significantly outperforming generic, un-augmented LLMs that are prone to hallucinating academic policies. The integration of the custom text normalization mapping specifically optimized for student dialects proved crucial in bridging the gap between colloquial queries and formal university documentation.

11.2 Conclusion and Future Work

This project successfully designed, implemented, and deployed an intelligent conversational agent tailored for the Data Science and Artificial Intelligence context. It meets the objective of providing instant, reliable 24/7 support while reducing the administrative burden on department staff.

Future Work:

- **Real-time Portal Integration:** Integrating the chatbot securely with the university's active databases (e.g., live course registration systems) to provide personalized, student-specific schedule information.

- **Faculty Expansion:** Scaling the vector database beyond a single department to encompass the entirety of the university's faculties.